在高并发场景下，服务限流是保障系统稳定性和可用性的关键手段。关于常见的限流算法（如令牌桶、漏桶）和限流技术方案，JavaGuide 的《服务限流详解》一文已有详尽介绍，本文不再赘述。

本文将重点介绍如何利用 **Redisson + 自定义 Lua 脚本**，结合 AOP (面向切面编程) 和自定义注解，来构建一个灵活、易用且支持分布式环境的限流解决方案。

何时选择分布式限流?

在讨论具体实现之前，我们需要明确分布式限流的适用场景。

以本项目为例，如果它始终是**单体应用**且只部署**单个实例**，那么引入基于 Redis 的分布式限流可能并非最优解。在这种单实例场景下，使用进程内的限流库，如 Google Guava 的 RateLimiter、Bucket4j 或 Resilience4j，通常是更轻量、高效和节省成本的选择。

然而，当应用需要**水平扩展**（即部署多个实例以承载更高流量）时，分布式限流就变得至关重要。想象一下，如果限制某个用户每秒只能访问 5 次，但在 3 个实例上各自使用内存限流器，用户实际可能达到 15 次/秒的访问速率，远超预期。**利用 Redis 作为共享的、集中式的状态存储，通过 Lua 脚本原子操作确保所有应用实例都遵循统一的限流规则，从而实现精确的全局速率控制。**

> **面试提示：** 如果面试官问及为何在单体项目中考虑使用分布式限流，务必能够清晰阐述是为未来的水平扩展做准备，或明确指出当前场景下更适合单机限流方案。理解方案的适用边界，避免留下技术选型不当的印象。

单机限流 vs 分布式限流

对比维度	单机限流（如 Guava RateLimiter）	分布式限流（如 Redis + Lua）
实现原理	进程内内存维护计数器	Redis 作为共享存储
适用场景	单实例应用	多实例集群部署
性能开销	极低（内存操作）	中等（网络 I/O）
数据一致性	实例间独立，无法协同	全局统一限流
运维成本	无需额外组件	需要 Redis 服务

整体架构设计

本项目结合 AOP 和自定义注解来实现基于 Redisson 的分布式限流，是业界广泛采用且推荐的一种模式。其优势在于：

1. **关注点分离 (AOP):** 限流逻辑本质上与核心业务逻辑解耦，属于典型的横切关注点。AOP 允许我们将限流的通用逻辑从业务方法中抽离出来，集中到一个切面类中管理。
2. **声明式使用 (Annotation):** 通过定义 `@RateLimit` 注解，开发者只需在需要限流的方法上添加注解并配置参数即可。
3. **分布式支持:** 利用 Redis 作为中心存储，通过 Lua 脚本保证操作的原子性。
4. **Redis Cluster 兼容:** 通过 Hash Tag 确保所有限流 Key 落在同一个 Slot。

引入依赖

本项目使用 **Redisson** 作为 Redis 客户端。

添加依赖：

```
// Redisson 4.0 - Redis客户端 (Boot 4.0 compatible)
implementation "org.redisson:redisson-spring-boot-
starter:${libs.versions.redisson.get()}"
```

`application.yml` ** 中配置 Redis: **

```yaml
# application.yml
spring:
  # Redisson配置 (使用 spring.redis.redisson, 参考官方文档)
  redis:
    redisson:
      config: |
        singleServerConfig:
          address: "redis://${REDIS_HOST:localhost}:${REDIS_PORT:6379}"
          database: 0
          connectionMinimumIdleSize: 10
          connectionPoolSize: 64
          subscriptionConnectionMinimumIdleSize: 1
          subscriptionConnectionPoolSize: 50
```

限流注解设计

`@RateLimit` 注解是整个限流组件的入口，定义了限流的所有配置项。

```java
package interview.guide.common.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/**
 * 限流注解
 * 用于方法级别的限流控制，支持多维度组合限流
 *
 * @see RateLimitAspect
 */
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RateLimit {

    /**
     * 限流维度枚举
     */
    enum Dimension {
        /**
         * 全局限流：对所有请求统一限流
         */
        GLOBAL,
        /**
         * IP限流：按客户端IP地址限流
         */
        IP,
        /**
         * 用户限流：按用户ID限流
```

```
         */
        USER
    }

    /**
     * 限流维度配置
     * 支持多维度组合，只有所有维度都满足条件时才允许请求通过
     */
    Dimension[] dimensions() default {Dimension.GLOBAL};

    /**
     * 在指定时间窗口内允许的最大请求数
     * 例如：count = 10, interval = 1, timeUnit = SECONDS 表示每秒最多 10 次
     */
    double count();

    /**
     * 时间窗口大小，默认 1
     */
    long interval() default 1;

    /**
     * 时间单位，默认为秒
     */
    TimeUnit timeUnit() default TimeUnit.SECONDS;

    /**
     * 等待令牌的超时时间
     * 0 表示不等待，直接获取令牌，失败则拒绝
     */
    long timeout() default 0;

    /**
     * 降级方法名
     * 支持无参方法或与原方法参数列表一致的方法
     */
    String fallback() default "";

    /**
     * 时间单位枚举
     */
    enum TimeUnit {
        MILLISECONDS, SECONDS, MINUTES, HOURS, DAYS
    }
}
```

设计要点：

1. `count`** + `interval` + `timeUnit` 三参数设计**：比单一的 `permitsPerSecond` 更灵活。
2. `dimensions`** 数组：**支持多维度组合，所有维度**同时满足**才能通过。
3. `fallback`** 降级**：限流触发时可执行降级方法，而非直接抛异常。

AOP 切面实现

`RateLimitAspect` 是限流逻辑的核心，负责拦截注解、生成 Key、调用 Lua 脚本、处理结果。

```java
package interview.guide.common.aspect;

/**
 * 限流 AOP 切面
 * 实现基于滑动时间窗口实现的多维度原子限流
 */
@Slf4j
@Aspect
@Component
@RequiredArgsConstructor
public class RateLimitAspect {

    private final RedissonClient redissonClient;

    /**
     * Lua 脚本缓存
     */
    private static String LUA_SCRIPT;
    private String luaScriptSha;

    static {
        try {
            ClassPathResource resource = new
ClassPathResource("scripts/rate_limit.lua");
            LUA_SCRIPT = new String(resource.getContentAsByteArray(),
StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new RuntimeException("加载限流 Lua 脚本失败", e);
        }
    }

    /**
     * 初始化：预加载脚本到 Redis 提高性能
     */
    @jakarta.annotation.PostConstruct
    public void init() {
        this.luaScriptSha =
redissonClient.getScript(StringCodec.INSTANCE).scriptLoad(LUA_SCRIPT);
        log.info("限流 Lua 脚本加载完成, SHA1: {}", luaScriptSha);
    }

    /**
     * 环绕通知：拦截带 @RateLimit 注解的方法
     */
    @Around("@annotation(rateLimit)")
    public Object around(ProceedingJoinPoint joinPoint, RateLimit rateLimit)
throws Throwable {
        MethodSignature signature = (MethodSignature) joinPoint.getSignature();
        Method method = signature.getMethod();
        String className = method.getDeclaringClass().getSimpleName();
        String methodName = method.getName();

        // 1. 计算时间窗口（毫秒）
        long intervalMs = calculateIntervalMs(rateLimit.interval(),
rateLimit.timeUnit());

        // 2. 根据配置维度动态生成 Redis Keys
```

```
        List<String> keys = generateKeys(className, methodName,
rateLimit.dimensions());

        // 3. 调用 Lua 脚本执行原子限流
        RScript script = redissonClient.getScript(StringCodec.INSTANCE);

        // 准备参数
        List<Object> keysList = new ArrayList<>(keys);
        Object[] args = {
                String.valueOf(System.currentTimeMillis()), // ARGV[1]: 当前时间戳
                String.valueOf(1),                          // ARGV[2]: 申请令牌数
（默认1个）
                String.valueOf(intervalMs),                 // ARGV[3]: 时间窗口
                String.valueOf(rateLimit.count()),          // ARGV[4]: 最大令牌数
                UUID.randomUUID().toString()               // ARGV[5]: 请求唯一标
识
        };

        Object resultObj = script.evalSha(
                RScript.Mode.READ_WRITE,
                luaScriptSha,
                RScript.ReturnType.VALUE,
                keysList,
                args
        );

        // 将结果转换为 Long
        Long result = convertToLong(resultObj);

        // 4. 处理限流结果
        if (result == null || result == 0) {
            return handleRateLimitExceeded(joinPoint, rateLimit, keys);
        }

        // 5. 执行原方法
        return joinPoint.proceed();
    }

    /**
     * 计算时间窗口毫秒数
     */
    private long calculateIntervalMs(long interval, RateLimit.TimeUnit unit) {
        return switch (unit) {
            case MILLISECONDS -> interval;
            case SECONDS -> interval * 1000;
            case MINUTES -> interval * 60 * 1000;
            case HOURS -> interval * 3600 * 1000;
            case DAYS -> interval * 86400 * 1000;
        };
    }
}
```

执行流程图:

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
| 1.计算时间    |──▶| 2.生成Keys   |──▶| 3.调用Lua    |
|   窗口       |      | (Hash Tag)   |      |   脚本       |
```

```
└──────────────┘      └──────────────┘      └──────────────┘
                                                   │
                                                   ▼
                                            ┌──────────────┐
                                            │ 4.处理结果    │
                                            │ (0=拒绝/1=通过)│
                                            └──────────────┘
                                                   │
                                   ┌───────────────┴────────────────┐
                                   ▼                                ▼
                          ┌──────────────┐                  ┌──────────────┐
                          │ 5a.执行降级   │                  │ 5b.执行原方法 │
                          │   或抛异常    │                  │  proceed()   │
                          └──────────────┘                  └──────────────┘
```

关键点说明:

- **SHA 预加载**: 启动时将 Lua 脚本加载到 Redis，后续使用 SHA 调用，减少网络传输。
- **StringCodec**： 确保 Redisson 参数正确传递为字符串，避免类型转换问题。
- **Hash Tag**： `{className:methodName}` 确保 Redis Cluster 模式下 Key 在同一 Slot。

Redis Key 生成（Hash Tag 机制）

```java
/**
 * 生成限流键列表
 */
private List<String> generateKeys(String className, String methodName,
RateLimit.Dimension[] dimensions) {
    List<String> keys = new ArrayList<>();
    // 使用 {} 包含类名和方法名作为 Hash Tag
    // 确保该方法的所有限流 Key 落在同一个 Redis Slot
    // 从而适配 Redis Cluster 模式
    String hashTag = "{" + className + ":" + methodName + "}";
    String keyPrefix = "ratelimit:" + hashTag;

    for (RateLimit.Dimension dimension : dimensions) {
        switch (dimension) {
            case GLOBAL -> keys.add(keyPrefix + ":global");
            case IP -> keys.add(keyPrefix + ":ip:" + getClientIp());
            case USER -> keys.add(keyPrefix + ":user:" + getCurrentUserId());
        }
    }

    return keys;
}
```

> **Hash Tag 机制**: 在 Redis Cluster 中，数据通过 CRC16 算法分散到 16384 个 Slot。使用 `{...}` 包裹相同内容的 Key 会确保它们落在同一个 Slot，避免跨 Slot 操作错误。

Key 格式示例:

```
ratelimit:{ResumeController:uploadAndAnalyze}:global:value
ratelimit:{ResumeController:uploadAndAnalyze}:global:permits
ratelimit:{ResumeController:uploadAndAnalyze}:ip:192.168.1.100:value
ratelimit:{ResumeController:uploadAndAnalyze}:ip:192.168.1.100:permits
```

IP 获取逻辑（支持代理）

```java
/**
 * 获取客户端真实 IP
 * 处理 X-Forwarded-For 头，支持代理服务器场景
 */
private String getClientIp() {
    ServletRequestAttributes attributes = (ServletRequestAttributes)
RequestContextHolder.getRequestAttributes();
    if (attributes == null) {
        return "unknown";
    }

    HttpServletRequest request = attributes.getRequest();
    String ip = request.getHeader("X-Forwarded-For");

    if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
        ip = request.getHeader("X-Real-IP");
    }
    if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
        ip = request.getHeader("Proxy-Client-IP");
    }
    if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
        ip = request.getHeader("WL-Proxy-Client-IP");
    }
    if (ip == null || ip.isEmpty() || "unknown".equalsIgnoreCase(ip)) {
        ip = request.getRemoteAddr();
    }

    // 处理多个 IP 的情况（X-Forwarded-For 可能包含多个 IP）
    if (ip != null && ip.contains(",")) {
        ip = ip.split(",")[0].trim();
    }

    return ip != null ? ip : "unknown";
}
```

获取优先级：`X-Forwarded-For` → `X-Real-IP` → `Proxy-Client-IP` → `WL-Proxy-Client-IP` → `RemoteAddr`

降级处理

```java
/**
 * 处理限流超出情况
 */
private Object handleRateLimitExceeded(ProceedingJoinPoint joinPoint, RateLimit
rateLimit, List<String> keys)
        throws Throwable {
    String methodName = joinPoint.getSignature().getName();
```

```
    // 如果配置了降级方法，则调用降级方法
    if (rateLimit.fallback() != null && !rateLimit.fallback().isEmpty()) {
        try {
            Method fallbackMethod = findFallbackMethod(joinPoint,
rateLimit.fallback());
            if (fallbackMethod != null) {
                log.debug("限流触发，执行降级方法: {}.{} -> {}",
                        joinPoint.getTarget().getClass().getSimpleName(),
                        methodName,
                        rateLimit.fallback());
                // 如果降级方法有参数，传入原方法的参数
                if (fallbackMethod.getParameterCount() > 0) {
                    return fallbackMethod.invoke(joinPoint.getTarget(),
joinPoint.getArgs());
                } else {
                    return fallbackMethod.invoke(joinPoint.getTarget());
                }
            }
        } catch (Exception e) {
            log.error("降级方法执行失败: {}", rateLimit.fallback(), e);
        }
    }

    // 没有降级方法或降级失败，抛出限流异常
    log.debug("限流触发，拒绝请求: keys={}, count={} per {} {}",
            keys, rateLimit.count(), rateLimit.interval(),
rateLimit.timeUnit());
    throw new RateLimitExceededException("请求过于频繁，请稍后再试");
}
```

Lua 脚本实现

Lua 脚本是限流逻辑的核心，确保多维度限流的原子性。

完整脚本

```
-- 原子化多维度限流脚本
-- 基于滑动时间窗口实现的多维度原子限流
-- 只有所有维度都满足条件时才扣减令牌，确保原子性

-- 参数说明:
-- KEYS[1..N]: 限流维度键列表
-- ARGV[1]: 当前时间戳（毫秒）
-- ARGV[2]: 申请令牌数
-- ARGV[3]: 时间窗口（毫秒）
-- ARGV[4]: 最大令牌数（窗口内允许的总数）
-- ARGV[5]: 请求唯一标识

local now_ms = tonumber(ARGV[1])
local permits = tonumber(ARGV[2])
local interval = tonumber(ARGV[3])
local max_tokens = tonumber(ARGV[4])
local request_id = ARGV[5]

-- ========== 第一阶段: 预检查 ==========
-- 检查所有维度是否有足够令牌
```

```lua
for i, key in ipairs(KEYS) do
    local value_key = key .. ":value"
    local permits_key = key .. ":permits"

    -- 初始化 value_key（如果不存在）
    if redis.call("exists", value_key) == 0 then
        redis.call("set", value_key, max_tokens)
    end

    -- 回收过期令牌
    -- 清理过期的 permit 记录，并回收配额到 value_key
    local expired_values = redis.call("zrangebyscore", permits_key, 0, now_ms -
interval)
    if #expired_values > 0 then
        local expired_count = 0
        for _, v in ipairs(expired_values) do
            -- 优化解析逻辑：使用更高效的模式匹配
            local p = tonumber(string.match(v, ":(%d+)$"))
            if p then
                expired_count = expired_count + p
            end
        end

        -- 删除过期记录
        redis.call("zremrangebyscore", permits_key, 0, now_ms - interval)

        -- 回收配额
        if expired_count > 0 then
            local curr_v = tonumber(redis.call("get", value_key) or max_tokens)
            local next_v = math.min(max_tokens, curr_v + expired_count)
            redis.call("set", value_key, next_v)
        end
    end

    -- 核心检查：当前可用令牌是否足够
    local current_val = tonumber(redis.call("get", value_key) or max_tokens)
    if current_val < permits then
        -- 任何一个维度配额不足，直接返回失败
        return 0
    end
end

-- ========== 第二阶段：扣减 ==========
-- 只有所有维度都通过后才执行
for i, key in ipairs(KEYS) do
    local value_key = key .. ":value"
    local permits_key = key .. ":permits"

    -- 记录本次令牌分配（格式：request_id:permits）
    local permit_record = request_id .. ":" .. permits
    redis.call("zadd", permits_key, now_ms, permit_record)

    -- 扣减令牌
    local current_v = tonumber(redis.call("get", value_key) or max_tokens)
    redis.call("set", value_key, current_v - permits)

    -- 设置过期时间，确保过期令牌能被正常回收 (窗口的2倍，至少1秒)
    local expire_time = math.ceil(interval * 2 / 1000)
```

```
    if expire_time < 1 then expire_time = 1 end
    redis.call("expire", value_key, expire_time)
    redis.call("expire", permits_key, expire_time)
end

-- 成功获取所有维度的令牌
return 1
```

Redis 数据结构

为了实现精准限流，每个维度使用了两种数据结构进行配合：

- **String (`{key}:value`)**：
  - **作用**：实时计数器。
  - **优点**：读写 O(1)，快速反馈当前额度。
- **Sorted Set (`{key}:permits`)**：
  - **作用**：时间轴流水账，记录每个请求的权重（令牌数）。
  - **Member 设计**：`request_id:permits`。这里的 `request_id` 极其关键，因为 ZSet 会覆盖相同的 Member。如果不加 UUID，同一毫秒内的多个请求将被合并，导致限流失效。
  - **Score 设计**：使用毫秒时间戳，方便通过 `zrangebyscore` 进行范围检索和删除。

使用示例

基础用法

```
@PostMapping("/api/resumes/upload")
@RateLimit(dimensions = {Dimension.GLOBAL, Dimension.IP}, count = 5)
public Result<Map<String, Object>> uploadAndAnalyze(@RequestParam("file")
MultipartFile file) {
    // 业务逻辑
    Map<String, Object> result = uploadService.uploadAndAnalyze(file);
    return Result.success(result);
}
```

上述配置表示：

- **全局维度**：每秒最多 5 次请求
- **IP 维度**：每个 IP 每秒最多 5 次请求
- 两个条件**同时满足**才能通过（AND 逻辑）

带降级方法

```
@PostMapping("/api/resumes/{id}/reanalyze")
@RateLimit(dimensions = {Dimension.GLOBAL}, count = 2, fallback =
"reanalyzeFallback")
public Result<Void> reanalyze(@PathVariable Long id) {
    uploadService.reanalyze(id);
    return Result.success(null);
}

// 降级方法（无参或参数一致）
private Result<Void> reanalyzeFallback() {
    return Result.error("系统繁忙，请稍后再试");
}
```

多种时间单位

```
// 每分钟 100 次
@RateLimit(count = 100, interval = 1, timeUnit = TimeUnit.MINUTES)

// 每小时 1000 次
@RateLimit(count = 1000, interval = 1, timeUnit = TimeUnit.HOURS)

// 每 500 毫秒 1 次
@RateLimit(count = 1, interval = 500, timeUnit = TimeUnit.MILLISECONDS)
```

与 Redisson 内置 RRateLimiter 的区别

特性	自定义 Lua 实现	Redisson RRateLimiter
多维度支持	原生支持，组合 AND 逻辑	需要多次调用
Redis Cluster	Hash Tag 自动适配	需要额外处理
定制化	完全可控	依赖内部实现
复杂度	需要维护 Lua 脚本	开箱即用
性能	SHA 预加载优化	预编译脚本

如果项目只需要简单的单维度限流且不使用 Redis Cluster，直接使用 `RRateLimiter` 是更简单的选择：

```
RRateLimiter rateLimiter = redisson.getRateLimiter("myLimiter");
rateLimiter.trySetRate(RateType.OVERALL, 5, 1, RateIntervalUnit.SECONDS);
if (rateLimiter.tryAcquire(1)) {
    // 允许请求
}
```

测试验证

集成测试

```java
@Test
@DisplayName("验证限流：令牌充足时允许，耗尽时拒绝")
void testRateLimit() {
    // 初始化 2 个令牌
    redissonClient.getBucket(valueKey, StringCodec.INSTANCE).set("2");

    // 前两次成功
    assertEquals(1L, executeLuaScript(keyPrefix, maxCount));
    assertEquals(1L, executeLuaScript(keyPrefix, maxCount));

    // 第三次被拒绝
    assertEquals(0L, executeLuaScript(keyPrefix, maxCount));
}
```

压测工具推荐

使用 **wrk** 进行压测：

```bash
# 基础压测
wrk -t4 -c100 -d30s http://localhost:8080/api/resumes/upload

# 参数说明
# -t4: 4 个工作线程
# -c100: 100 个并发连接
# -d30s: 持续 30 秒
```

总结

核心组件：

组件	说明
`@RateLimit`	限流注解，支持多维度配置
`RateLimitAspect`	AOP 切面，拦截注解并执行限流逻辑
Lua 脚本	原子化限流算法，支持令牌回收
Redisson	Redis 客户端，提供脚本执行能力

技术亮点：

1. **灵活的时间窗口**：支持秒/分/时/天/毫秒多种时间单位
2. **多维度组合**：支持 GLOBAL/IP/USER 多种维度的 AND 组合
3. **Redis Cluster 兼容**：通过 Hash Tag 确保 Key 分布在同一 Slot
4. **高性能**：Lua 脚本 SHA 预加载，减少网络传输
5. **降级支持**：可配置降级方法，提升用户体验
6. **代理兼容**：正确处理 X-Forwarded-For 等代理场景

希望本文能帮助你理解分布式限流的实现原理，并在实际项目中应用！完整代码可参考项目源码中的以下文件：

- `common/annotation/RateLimit.java` - 限流注解定义
- `common/aspect/RateLimitAspect.java` - AOP 切面实现
- `common/exception/RateLimitExceededException.java` - 限流异常

- `resources/scripts/rate_limit.lua` - Lua 限流脚本
- `modules/resume/ResumeController.java` - 使用示例